

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ  
ФЕДЕРАЦИИ**

Федеральное государственное автономное образовательное учреждение  
высшего профессионального образования  
**НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТОМСКИЙ ПОЛИТЕХНИЧЕСКИЙ  
УНИВЕРСИТЕТ**

Инженерная школа информационных технологий и робототехники

Направление 15.04.06 «Мехатроника и робототехника»

Отделение автоматизации и робототехники

Знакомство с лабораторной базой. Изучение и настройка GPIO. Применение  
библиотеки CMSIS при программировании микроконтроллеров

Отчет по лабораторной работе № 1  
по дисциплине «Микропроцессоры и микроконтроллеры»

Выполнил ст. гр. 8ЕМ51 \_\_\_\_\_ Якушев Н.Е.

Проверил старший \_\_\_\_\_ Поберезкин Н.И.  
преподаватель

Должность

Подпись

Дата

Фамилия И.О.

## Содержание

|                                                              |    |
|--------------------------------------------------------------|----|
| Цель работы.....                                             | 3  |
| Задание.....                                                 | 3  |
| Ход работы.....                                              | 3  |
| 1. Подготовка к выполнению задания.....                      | 3  |
| 1.1. Подготовка инфраструктуры.....                          | 3  |
| 1.2. Работа с встроенным светодиодом и тактовой кнопкой..... | 4  |
| 2. Выполнение задания.....                                   | 6  |
| 2.1. Назначение пинов.....                                   | 6  |
| 2.2. Разработка алгоритма.....                               | 7  |
| 3. Тестирование алгоритмов.....                              | 12 |
| 3.1. Результаты выполнения задания.....                      | 12 |
| 3.2. Запуск в ПО MCUViewer.....                              | 12 |
| Подведение итогов.....                                       | 15 |
| Приложение А Программная часть для выполнения задания.....   | 16 |
| Приложение Б Принципиальная схема подключения элементов..... | 23 |

## Цель работы

Знакомство с лабораторной базой. Изучение и настройка *GPIO*. Применение библиотеки *CMSIS* при программировании микроконтроллеров.

## Задание

Собрать схему с включением 3-х светодиодов и 4-х кнопок. Базовый функционал кнопок, указанный ниже, меняется кнопкой 4:

- 1) При удержании, включает зеленый светодиод на отладочной плате.
- 2) При удержании, включает синий светодиод на отладочной плате.
- 3) При удержании, включает красный светодиод на отладочной плате.
- 4) При нажатии меняет функционал у кнопок смещая цвет светодиода

вниз по списку.

То есть, при первом нажатии 4-ой кнопки, кнопка-1 будет включать красный светодиод, кнопка-2 зелёный, кнопка-3 синий. Каждое третье нажатие на кнопку 4 возвращает функционал всех кнопок к базовому.

## Ход работы

### 1. Подготовка к выполнению задания

#### 1.1. Подготовка инфраструктуры

Для всех дальнейших работ используется отладочная плата *Nucleo-64* на базе микроконтроллера *STM32F446RET6*.

На рисунке 1 представлен QR-код на *GitHub* проекта.



Рисунок 1 – QR-код *GitHub* проекта

С использованием методического материала “Подготовка инфраструктуры.pdf” была настроена среда для работы с микроконтроллером. На рисунке 2 представлен вид настроенной файловой инфраструктуры.

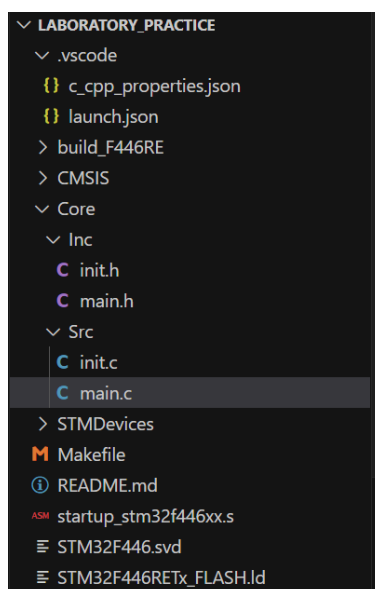


Рисунок 2 – Файловая инфраструктура проекта

## 1.2. Работа с встроенным светодиодом и тактовой кнопкой

Для проверки правильной настройки проекта было выполнено задание с зажиганием светодиода при нажатии тактовой кнопки. В нашем случае зеленый светодиод имеет пин *PA5*, а тактовая кнопка – *PC13*.

Для работы со светодиодом и тактовой кнопкой была выполнена следующая цепочка действий:

- включение тактирования *GPIO* на периферию ***GPIOAEN*** (светодиод) и ***GPIOCEN*** (тактовая кнопка),
- задание режима работы порта (для светодиода). Необходимо включить режим “01: *General Purpose Output mode*” (режим вывода сигнала на порт микроконтроллера),
- задание режима вывода порта (для светодиода). Не требует изменения, так как задан необходимый режим “0: *Output push-pull (reset state)*” (двухтактный выход).
- задание скорости работы пинов порта (для светодиода). Необходимо включить режим “01: *Medium speed*” (диапазон частот от 10 до 50 МГц),
- задание настройки подтяжки (для светодиода). Не требует изменения, так как задан необходимый режим “00: *No pull-up, pull-down*” (нет подтяжки к питанию и стяжки на общий провод).
- задание цикла *while(1)* с проверкой состояния тактовой кнопки (на данной схеме инвертирована). При нажатии на кнопку светодиод загорается, иначе светодиод не горит.

Код с реализацией представлен в листинге 1.

Листинг 1 – Программный код для работы со светодиодом и тактовой кнопкой

```
1. #include "stdint.h"
2. int main(void)
3. {
4.     // Включение тактирования на GPIOCEN
5.     *(uint32_t*)(0x40023800UL + 0x30UL) |= (1 << 2);
6.     // Включение тактирования на GPIOAEN
7.     *(uint32_t*)(0x40023800UL + 0x30UL) |= (1 << 0);
8.     *(uint32_t*)(0x40020000UL + 0x00UL) |= (1 << 10); // Режим порта
9.     // Input mode (00). Режим считывания сигнала с порта
        микроконтроллера.
10.    // General Purpose Output mode (01). Режим вывода сигнала на порт
        микроконтроллера
11.    // Alternate function mode (10). Режим альтернативных функций.
12.    // Analog mode (11). Аналоговый режим.
13.    *(uint32_t*)(0x40020000UL + 0x04UL) |= 0x00; // настройка режима порта
14.    // Output Push-Pull (PP) 0 – Стандартный двухтактный выход.
15.    // Output Open-Drain (OD) 1 – Выход с открытым стоком.
16.    *(uint32_t*)(0x40020000UL + 0x08UL) |= (1 << 10); // скорость одного
        бита
17.    // 00: Low speed / Низкая скорость – диапазон частот от 2 до 8 МГц
18.    // 01: Medium speed / Средняя скорость – диапазон частот от 10 до 50
        МГц
19.    // 10: High speed / Высокая скорость – диапазон частот от 25 до 100 МГц
20.    // 11: Very high speed / Очень высокая скорость – диапазон частот от
        42,5 до 180 МГц
21.    *(uint32_t*)(0x40020000UL + 0x0CUL) |= (0x00); // настройка подтяжки
22.    // 00: No pull-up, pull-down – Нед подтяжки к питанию и стяжки на общий
        провод.
23.    // 01: Pull-up – Подтяжка к положительному напряжению через резистор.
24.    // 10: Pull-down – Стяжка к общему проводу через резистор.
25.    // 11: Reserved – Зарезервированное состояние, не используется в
        настройке
26.
27.    while(1) {
28.        // Проверка состояния тактовой кнопки
29.        if((*uint32_t*)(0x40020800UL + 0x10UL) & 0x2000UL) != 0) {
30.            // Выключение светодиода
31.            *(uint32_t*)(0x640020000UL + 0x18UL) |= (1 << 21);
32.        }
33.        else
34.        {
35.            // Включение светодиода
```

```

36.      *(uint32_t*)(0x40020000UL + 0x18UL) |= (1 << 5);
37.    }
38.  }
39. }

```

После сборки проекта и его прошивки в микроконтроллер, на отладочной плате зелёной светодиод не горит. После нажатия тактовой кнопки светодиод загорелся. Соответственно, настройка и реализация проекта с адресацией были выполнены корректно.

## 2. Выполнение задания

### 2.1. Назначение пинов

По описанию задания необходимо реализовать работу 6-ти светодиодов и 4-ых кнопок. Для этого на рисунке 3 приведена схема отладочной платы.

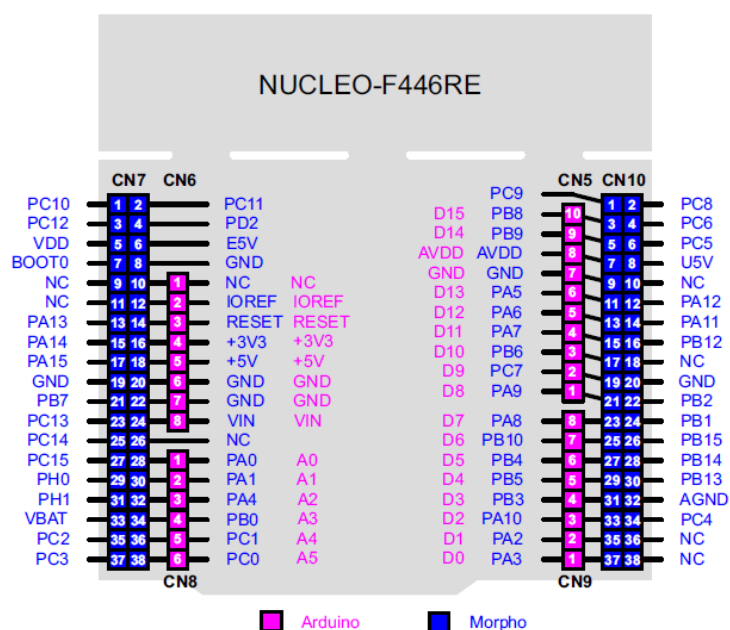


Рисунок 3 – Схема отладочной платы с обозначением пинов

Исходя из схемы на рисунке 3, были выбраны необходимые пины для выполнения задания, которые приведены в таблице 1. Так как в наличие не было синего светодиода, заменим его на жёлтый.

Таблица 1 – Выбранные пины для выполнения задания

| Элемент схемы                    | Назначенный пин |
|----------------------------------|-----------------|
| Внешние светодиоды               |                 |
| Светодиод №1 (зелёный светодиод) | PA0             |

|                                        |             |
|----------------------------------------|-------------|
| Светодиод №2 (жёлтый светодиод)        | <i>PA1</i>  |
| Светодиод №3 (красный светодиод)       | <i>PA4</i>  |
| Внешние кнопки                         |             |
| Кнопка №1 (включение светодиода)       | <i>PB10</i> |
| Кнопка №2 (включение светодиода)       | <i>PB5</i>  |
| Кнопка №3 (включение светодиода)       | <i>PB0</i>  |
| Кнопка №4 (переключение режима работы) | <i>PC13</i> |

Для работы с внешними светодиодами был порт *GPIOA* для работы с внешними кнопками – *GPIOB*.

## 2.2. Разработка алгоритма

При разборе задания можно выделить следующую последовательность действий:

- при первом запуске кода первая кнопка отвечает за зелёный светодиод, вторая кнопка отвечает за жёлтый светодиод, третья кнопка отвечает за красный светодиод (при нажатии на кнопку определенный светодиод загорается),
- при нажатии на четвертую кнопку режим работы меняется со сдвигом вниз: первая кнопка отвечает теперь отвечает за красный светодиод, вторая кнопка отвечает за зелёный светодиод, третья кнопка отвечает за жёлтый,
- после третьего нажатия режим полностью идентичен первому пункту.

На рисунке 4 представлена блок-схема работы основного алгоритма “*int main(void)*”. На рисунке 5 представлена блок-схема работы функции настройки периферии *void GPIO\_Init(void)*. На рисунке 6 представлена блок-схема работы функции смены режима *void GPIO\_MODE\_SELECTION(void)*. На рисунке 7 представлена блок-схема работы функции управления светодиодами *void LED\_STATE(void)*.

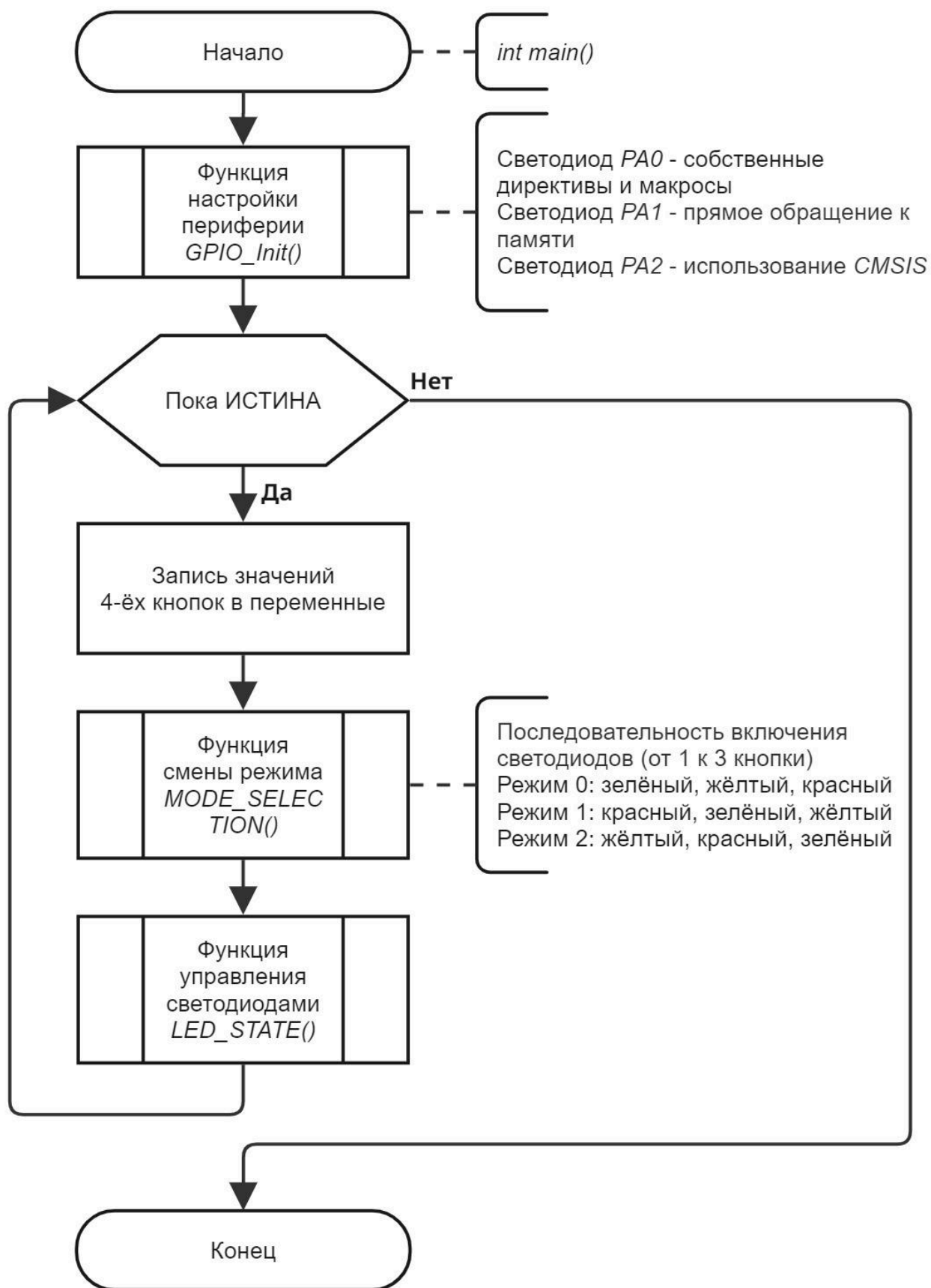


Рисунок 4 – Блок-схема работы основного цикла



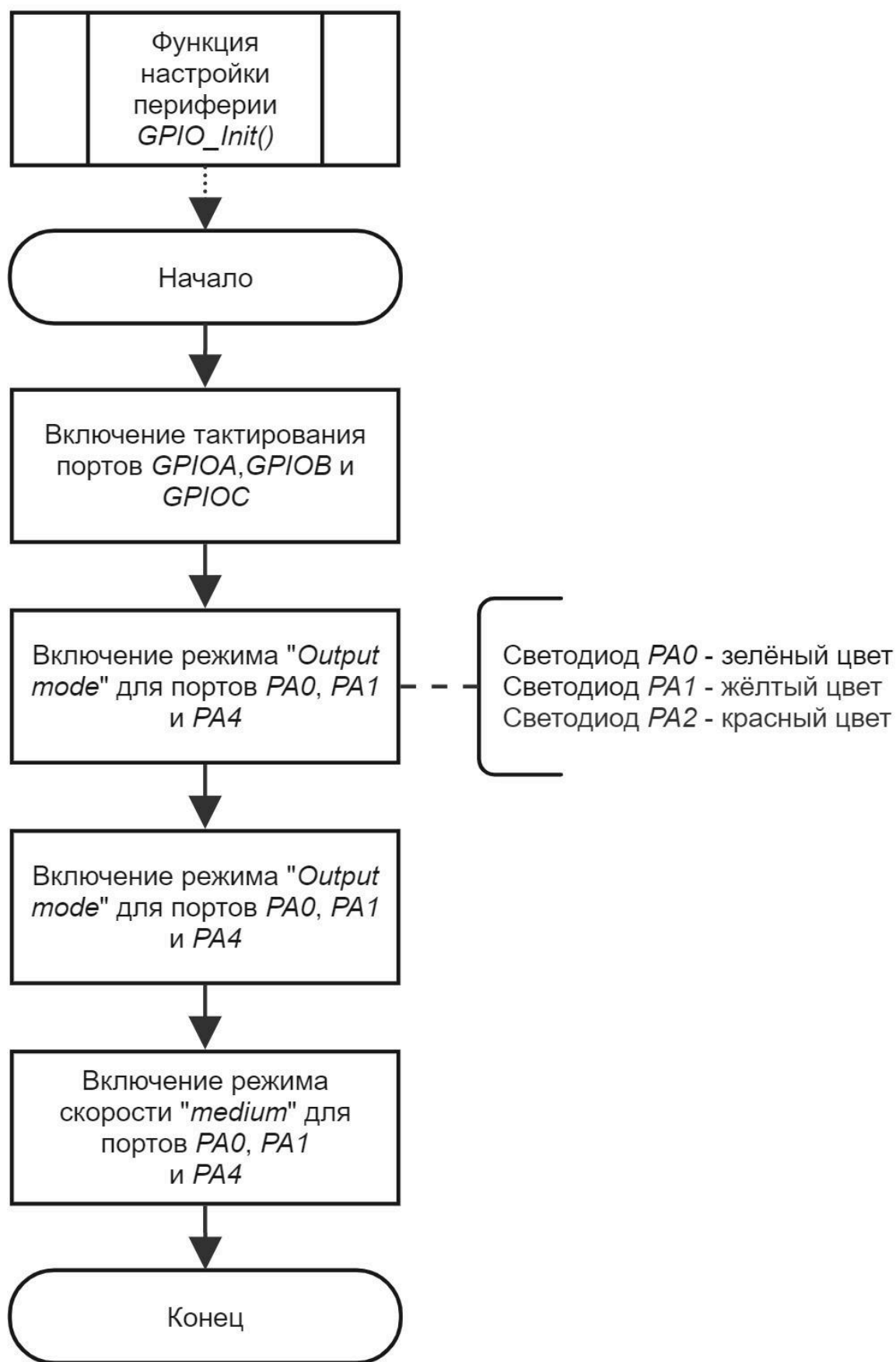


Рисунок 5 – Блок-схема работы функции первичной настройки периферии

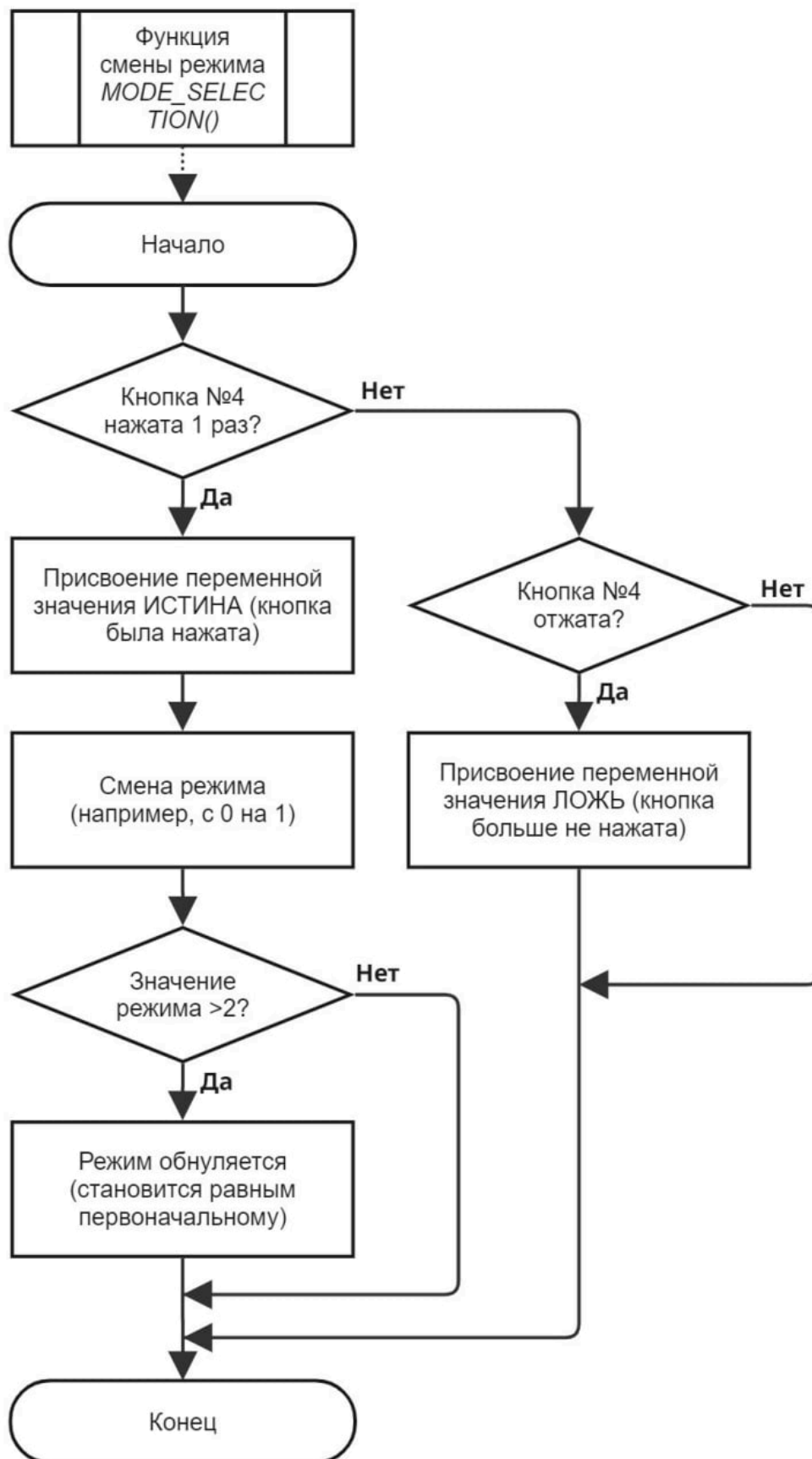


Рисунок 6 – Блок-схема работы функции смены режима

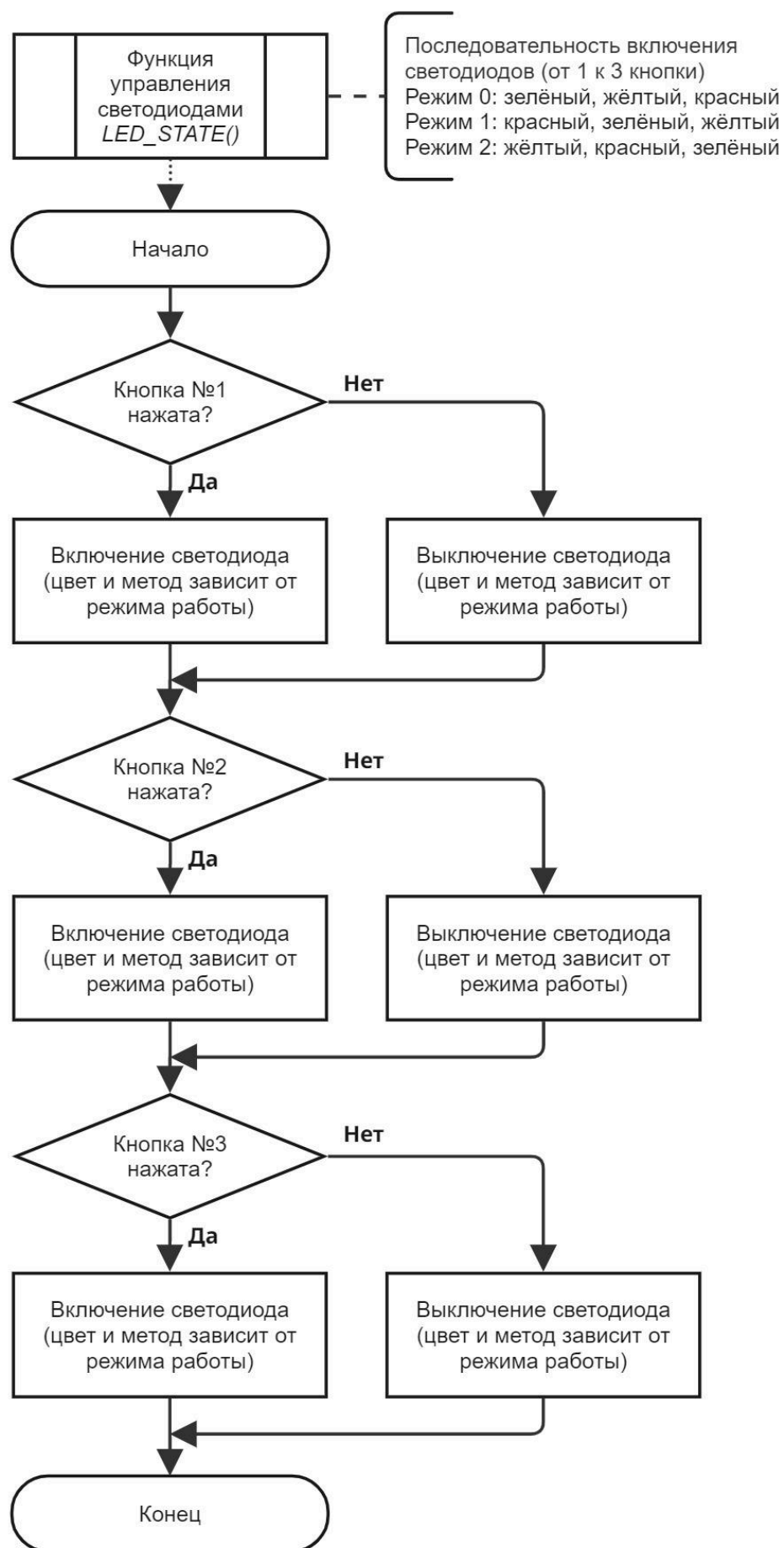


Рисунок 7 – Блок-схема работы управления светодиодами

### 3. Тестирование алгоритмов

#### 3.1. Результаты выполнения задания

На рисунке 8 можно найти видео формата *.gif* для демонстрации работы.



Рисунок 8 – QR-код видео демонстрации работы

Для тестирования алгоритма была собрана схема на макетной плате и написан программный код по ранее созданным алгоритмам. При написании кода были использованы функции для улучшения читаемости кода. Программная часть представлена в приложении А. Принципиальная схема подключения компонентов представлена в приложении Б.

#### 3.2. Запуск в ПО *MCUViewer*

Для визуализации работы программного кода с макетной платой было использовано ПО *MCUViewer* (аналог *STMStudio*). Для наглядности в программный код были добавлены переменные, отвечающие за состояния светодиодов (0 - выключен, 1 - включён). Вид макетной платы представлен на рисунке 8. Результаты работы представлены на рисунках 9-11. На рисунке 12 представлен график переключения режимов при нажатии 4 кнопки.

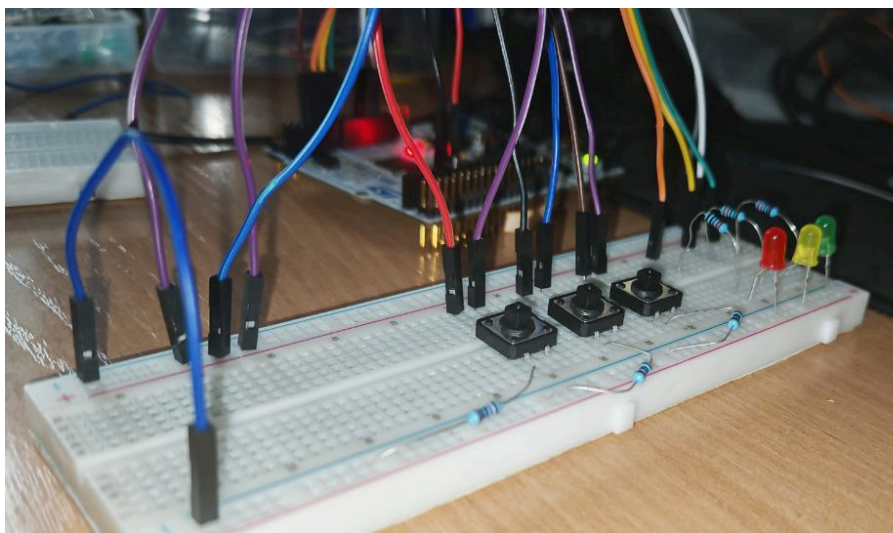


Рисунок 9 – Вид макетной платы



Рисунок 10 – Графики работы 0-го режима (стандарт)



Рисунок 11 – Графики работы 1-го режима



Рисунок 12 – Графики работы 2-го режима

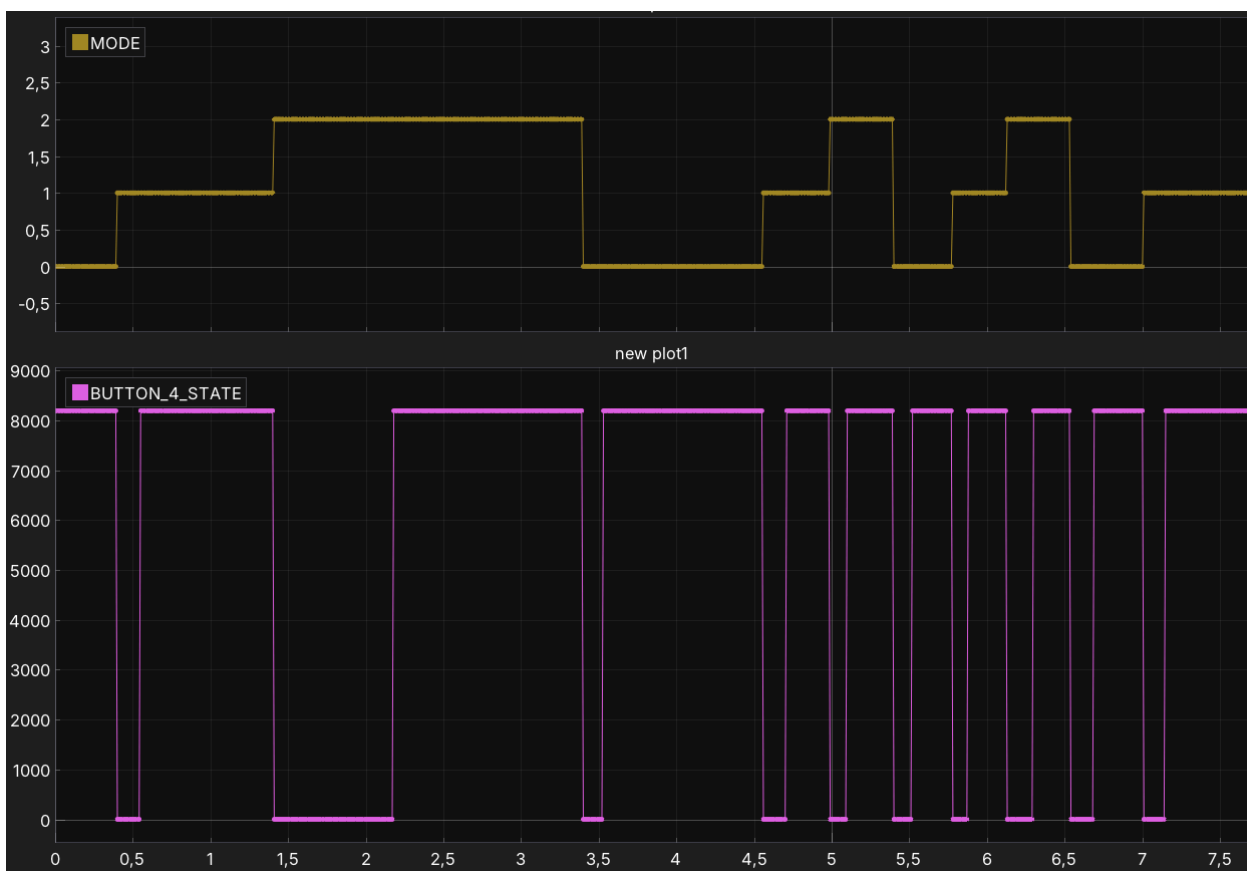


Рисунок 13 – График переключения режимов

### **Подведение итогов**

В результате лабораторной работы была подготовлена инфраструктура микроконтроллера STM32F446RE с отладочной платой Nucleo-64. При подготовке к выполнению задания была проведена работа по управлению встроенным светодиодом с помощью встроенной кнопки двумя способами: использование прямого обращения к памяти; использование собственных директив и макросов.

С использованием Reference Manual была произведена настройка периферии GPIOA, GPIOB и GPIOC. Был разработан алгоритмы для решения задания по варианту, после чего была собрана макетная плата и написан программный код на языке C. Было произведено тестирование разработанного программного кода в ПО MCUViewer.

В итоге тестирования функционал разработанного кода соответствует описанию задания и разработанному алгоритму.

Приложение А  
Программная часть для выполнения задания

Листинг А.1 – программный код файла *main.c*

```
1.  #include <init.h>
2.
3.  uint32_t BUTTON_1_STATE = 0; // состояние 1-ой внешней кнопки
4.  uint32_t BUTTON_2_STATE = 0; // состояние 2-ой внешней кнопки
5.  uint32_t BUTTON_3_STATE = 0; // состояние 3-ой внешней кнопки
6.  uint32_t BUTTON_4_STATE = 0; // состояние 4-ой встроенной кнопки
7.  uint8_t flag = 0;           // проверка на единичное нажатие 4-ой кнопки
8.
9.  uint8_t MODE = 0;          // режим работы
10. // режим 0: зелёный, жёлтый, красный
11. // режим 1: красный, зелёный, жёлтый
12. // режим 2: жёлтый, красный, зелёный
13.
14. // массив для включения светодиодов
15. const uint32_t MODE_SET[3][3] =
16. {
17.     {GPIO_BSRR_BS0, GPIO_BSRR_BS1, GPIO_BSRR_BS4},
18.
19.     {GPIO_BSRR_BS4, GPIO_BSRR_BS0, GPIO_BSRR_BS1},
20.
21.     {GPIO_BSRR_BS1, GPIO_BSRR_BS4, GPIO_BSRR_BS0}
22. };
23.
24. // массив для выключения светодиодов
25. const uint32_t MODE_RESET[3][3] =
26. {
27.     {GPIO_BSRR_BR0, GPIO_BSRR_BR1, GPIO_BSRR_BR4},
28.
29.     {GPIO_BSRR_BR4, GPIO_BSRR_BR0, GPIO_BSRR_BR1},
30.
31.     {GPIO_BSRR_BR1, GPIO_BSRR_BR4, GPIO_BSRR_BR0}
32. };
33.
34. // смена режима по состоянию 4 кнопки
35. void MODE_SELECTION(void)
36. {
37.     if (BUTTON_4_STATE == 0 && flag == 0)
38.     {
39.         flag = 1;
40.         MODE++;
41.         if (MODE > 2)
42.         {
43.             MODE = 0;
44.         }
45.     }
```



```

46.     else if (BUTTON_4_STATE != 0 && flag) // проверка зажатия
47.     {
48.         flag = 0;
49.     }
50. }
51.
52. // функционал кнопок для режима №0
53. void LED_MODE0(void)
54. {
55.     // включение 1-го в списке светодиода (по режиму)
56.     if (BUTTON_1_STATE != 0)
57.     {
58.         // PA0
59.         SET__BIT(GPIOA_BSRR, GPIOA_BSRR_PIN0_SET);
60.         //SET_BIT(GPIOA->BSRR, MODE_SET[MODE][0]);
61.     }
62.     else
63.     {
64.         SET__BIT(GPIOA_BSRR, GPIOA_BSRR_PIN0_RESET);
65.         //SET_BIT(GPIOA->BSRR, MODE_RESET[MODE][0]);
66.     }
67.
68.     // включение 2-го в списке светодиода (по режиму)
69.     if (BUTTON_2_STATE != 0)
70.     {
71.         // PA1
72.         *(uint32_t*)(0x640020000UL + 0x18UL) |= (1 << 1);
73.         //SET_BIT(GPIOA->BSRR, MODE_SET[MODE][1]);
74.     }
75.     else
76.     {
77.         *(uint32_t*)(0x640020000UL + 0x18UL) |= (1 << 17);
78.         //SET_BIT(GPIOA->BSRR, MODE_RESET[MODE][1]);
79.     }
80.
81.     // включение 1-го в списке светодиода (по режиму)
82.     if (BUTTON_3_STATE != 0)
83.     {
84.         // PA4
85.         SET_BIT(GPIOA->BSRR, MODE_SET[MODE][2]);
86.     }
87.     else
88.     {
89.         SET_BIT(GPIOA->BSRR, MODE_RESET[MODE][2]);
90.     }
91.
92. }
93.
94. // функционал кнопок для режима №1

```

```

95. void LED_MODE1(void)
96. {
97.     // включение 1-го в списке светодиода (по режиму)
98.     if (BUTTON_1_STATE != 0)
99.     {
100.         SET_BIT(GPIOA->BSRR, MODE_SET[MODE][0]);
101.     }
102.     else
103.     {
104.         SET_BIT(GPIOA->BSRR, MODE_RESET[MODE][0]);
105.     }
106.
107.     // включение 2-го в списке светодиода (по режиму)
108.     if (BUTTON_2_STATE != 0)
109.     {
110.         // PA0
111.         SET__BIT(GPIOA_BSRR, GPIOA_BSRR_PIN0_SET);
112.         //SET_BIT(GPIOA->BSRR, MODE_SET[MODE][0]);
113.     }
114.     else
115.     {
116.         SET__BIT(GPIOA_BSRR, GPIOA_BSRR_PIN0_RESET);
117.         //SET_BIT(GPIOA->BSRR, MODE_RESET[MODE][0]);
118.     }
119.
120.     // включение 3-го в списке светодиода (по режиму)
121.     if (BUTTON_3_STATE != 0)
122.     {
123.         // PA1
124.         *(uint32_t*)(0x640020000UL + 0x18UL) |= (1 << 1);
125.         //SET_BIT(GPIOA->BSRR, MODE_SET[MODE][1]);
126.     }
127.     else
128.     {
129.         *(uint32_t*)(0x640020000UL + 0x18UL) |= (1 << 17);
130.         //SET_BIT(GPIOA->BSRR, MODE_RESET[MODE][1]);
131.     }
132. }
133.
134. // функционал кнопок для режима №2
135. void LED_MODE2(void)
136. {
137.     // включение 1-го в списке светодиода (по режиму)
138.     if (BUTTON_1_STATE != 0)
139.     {
140.         // PA1
141.         *(uint32_t*)(0x640020000UL + 0x18UL) |= (1 << 1);
142.         //SET_BIT(GPIOA->BSRR, MODE_SET[MODE][1]);
143.     }

```

```

144. else
145. {
146.     *(uint32_t*)(0x640020000UL + 0x18UL) |= (1 << 17);
147.     //SET_BIT(GPIOA->BSRR, MODE_RESET[MODE][1]);
148. }
149.
150. // включение 2-го в списке светодиода (по режиму)
151. if (BUTTON_2_STATE != 0)
152. {
153.     SET_BIT(GPIOA->BSRR, MODE_SET[MODE][1]);
154. }
155. else
156. {
157.     SET_BIT(GPIOA->BSRR, MODE_RESET[MODE][1]);
158. }
159.
160. // включение 3-го в списке светодиода (по режиму)
161. if (BUTTON_3_STATE != 0)
162. {
163.     // PA0
164.     SET__BIT(GPIOA_BSRR, GPIOA_BSRR_PIN0_SET);
165.     //SET_BIT(GPIOA->BSRR, MODE_SET[MODE][0]);
166. }
167. else
168. {
169.     SET__BIT(GPIOA_BSRR, GPIOA_BSRR_PIN0_RESET);
170.     //SET_BIT(GPIOA->BSRR, MODE_RESET[MODE][0]);
171. }
172.}
173.
174.// включение и выключение светодиодов
175.void LED_STATE(void)
176.{
177.    if (MODE == 0) {
178.        LED_MODE0();
179.    }
180.    else if (MODE == 1) {
181.        LED_MODE1();
182.    }
183.    else if (MODE == 2) {
184.        LED_MODE2();
185.    }
186.
187.}
188.
189.int main(void)
190.{
191.    GPIO_Init(); // подключение портов GPIOA|GPIOB|GPIOC
192.

```

```

193. while(1) {
194.     // чтение состояния кнопки для управления светодиодами
195.     BUTTON_1_STATE = READ_BIT(GPIOB->IDR, GPIO_IDR_IDR_10);
196.     BUTTON_2_STATE = READ_BIT(GPIOB->IDR, GPIO_IDR_IDR_5);
197.     BUTTON_3_STATE = READ_BIT(GPIOB->IDR, GPIO_IDR_IDR_0);
198.
199.     // чтение состояния кнопки для смены режима (инвертировано)
200.     BUTTON_4_STATE = READ_BIT(GPIOC->IDR, GPIO_IDR_IDR_13);
201.
202.     // функция смены режима по состоянию 4 кнопки
203.     MODE_SELECTION();
204.
205.     // функция включения и выключения светодиодов
206.     LED_STATE();
207. }
208.}

```

Листинг А.2 – программный код файла *init.c*

```

1. #include <init.h>
2.
3. // настройка пина PA1
4. void PA1_MEMORY(void)
5. {
6.     // Включение тактирования на GPIOAEN
7.     *(uint32_t*)(0x40023800UL + 0x30UL) |= (1 << 0);
8.
9.     *(uint32_t*)(0x40020000UL + 0x00UL) |= (1 << 2); // Режим порта PA1
10.    // Input mode (00). Режим считывания сигнала с порта микроконтроллера.
11.    // General Purpose Output mode (01). Режим вывода сигнала на порт
12.    // Alternate function mode (10). Режим альтернативных функций.
13.    // Analog mode (11). Аналоговый режим.
14.
15.    *(uint32_t*)(0x40020000UL + 0x04UL) |= 0x00; // PA1
16.    // Output Push-Pull (PP) 0 – Стандартный двухтактный выход.
17.    // Output Open-Drain (OD) 1 – Выход с открытым стоком.
18.
19.    *(uint32_t*)(0x40020000UL + 0x08UL) |= (1 << 2); // скорость одного бита PA1
20.    // 00: Low speed / Низкая скорость – диапазон частот от 2 до 8 МГц
21.    // 01: Medium speed / Средняя скорость – диапазон частот от 10 до 50 МГц
22.    // 10: High speed / Высокая скорость – диапазон частот от 25 до 100 МГц
23.    // 11: Very high speed / Очень высокая скорость – частоты от 42,5 до 180 МГц
24.
25.    *(uint32_t*)(0x40020000UL + 0x0CUL) |= (0x00); // настройка подтяжки
26.    // 00: No pull-up, pull-down – Нед подтяжки к питанию и стяжки на общий провод.
27.    // 01: Pull-up – Подтяжка к положительному напряжению через резистор.
28.    // 10: Pull-down – Стяжка к общему проводу через резистор.
29.    // 11: Reserved – Зарезервированное состояние, не используется в настройке

```

```

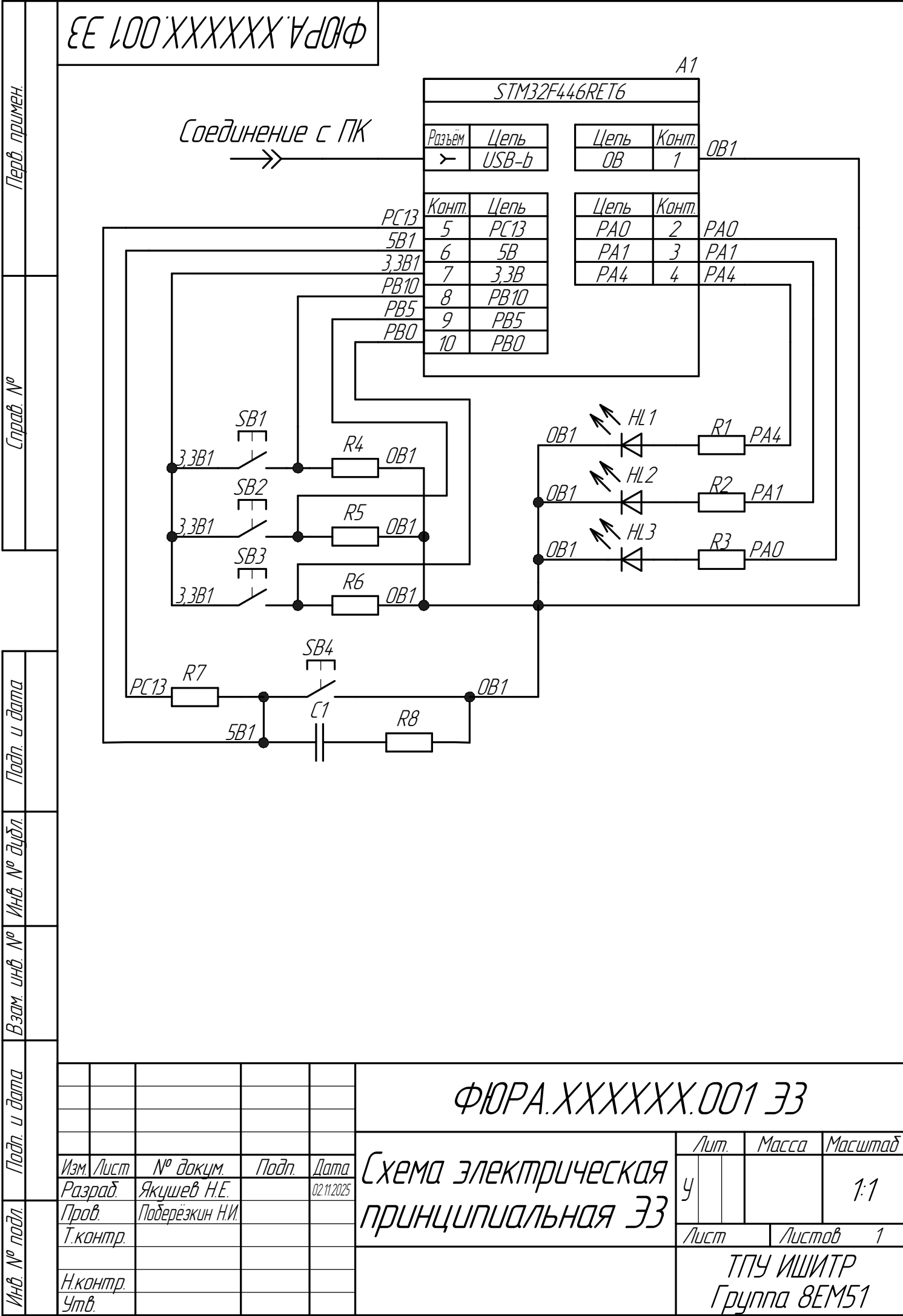
30.
31. }
32.
33. // настройка пина PA0
34. void PA0_MACRO(void)
35. {
36.     //RCC_GPIO_EN |= RCC_GPIOA_EN; // Включение тактирования портов GPIOA
37.     GPIOA_MODER |= GPIOA_MODE_PIN0_OUT; // Режим порта 0-го пина GPIOA
38.     GPIOA_OTYPER |= GPIOA_OTYPE_PIN0_PP; // Настройка на Push-Pull 0-го пина
39.     GPIOA_OSPEEDR |= GPIOA_OSPEED_PIN0_MID; // Скорость работы 0-го пина
40.     GPIOA_PUPDR |= GPIOA_PUPDR_PIN0_NOPUPD; // Настройка подтяжки
41. }
42.
43. // настройка пина PA4
44. void PA4_CMSIS(void)
45. {
46.     SET_BIT(GPIOA->MODER, GPIO_MODER_MODE4_0);
47.
48.     // Настройка на Push-Pull 4-го пина GPIOA
49.     CLEAR_BIT(GPIOA->OTYPER, GPIO_OTYPER_OT4);
50.
51.     // Настройка скорости работы 4-го пина GPIOA
52.     SET_BIT(GPIOA->OSPEEDR, GPIO_OSPEEDER_OSPEEDR4_0);
53.
54.     // Настройка подтяжки (отключение)
55.     SET_BIT(GPIOA->PUPDR, GPIO_PUPDR_PUPD4);
56. }
57.
58. void GPIO_Init(void)
59. {
60.     //Включение тактирования портов GPIOA|GPIOB|GPIOC
61.     SET_BIT(RCC->AHB1ENR, RCC_AHB1ENR_GPIOAEN |
62.             RCC_AHB1ENR_GPIOBEN |
63.             RCC_AHB1ENR_GPIOCEN);
64.
65.     // Прямое обращение к памяти (PA1)
66.     PA1_MEMORY();
67.
68.     // Собственные директивы и макросы (PA0)
69.     PA0_MACRO();
70.
71.     // Использование CMSIS (PA0)
72.     PA4_CMSIS();
73. }

```

Листинг А.3 – программный код файла *init.h*

```
1. #include <stm32f446xx.h>
2. #include <stm32f4xx.h>
3.
4. // для управления светодиодом на пине PA0
5. #define RCC_GPIOA_EN          (1 << 0)
6. #define GPIOA_MODE_PIN0_OUT   (1 << 0)
7. #define GPIOA_OTYPE_PIN0_PP   (0 << 0)
8. #define GPIOA_OSPEED_PIN0_MID (1 << 0)
9. #define GPIOA_PUPDR_PIN0_NOPUPD (0 << 0)
10. #define GPIOA_BSRR_PIN0_SET   (1 << 0)
11. #define GPIOA_BSRR_PIN0_RESET (1 << 16)
12.
13. #define RCC_GPIO_EN           (*(uint32_t*)(0x40023800UL + 0x30UL))
14. #define GPIOA_MODER           (*(uint32_t*)(0x40020000UL + 0x00UL))
15. #define GPIOA_OTYPER          (*(uint32_t*)(0x40020000UL + 0x04UL))
16. #define GPIOA_OSPEEDR         (*(uint32_t*)(0x40020000UL + 0x08UL))
17. #define GPIOA_PUPDR           (*(uint32_t*)(0x40020000UL + 0x0CUL))
18. #define GPIOA_BSRR            (*(uint32_t*)(0x40020000UL + 0x18UL))
19.
20. #define SET__BIT(REG, BIT)     (REG |= BIT)
21.
22. void GPIO_Init(void);
```

Приложение Б  
Принципиальная схема подключения элементов



ФЮРА.XXXXXXX.001 ЭЗ

Копировал